

;Dick Cappels' project pages projects@cappels.org - you should be able to copy and paste this text into an assembler file.
 ;©2003 Richard Cappels All Rights Reserved. No liability for results of use is assumed. See disclaimer on
 ;web site. email projects@cappels.org

;Use of information presented on this page is for personal, nonprofit educational
 ;and noncommercial use only. This material (including object files) is copyrighted
 ;by Richard Cappels and may not be republished or used directly for commercial
 ;purposes without explicit permission. For commercial license, [click HERE](#).



```
;
;HOME
;*****
;2313 serial to Sekio Epson 1216 display panel interface program
;*****

.include "2313def.inc"

;Driver for Seiko 1216 LCD display. 128 X 64, with separate drivers for left and right
;halves of the screen. See the specification and app. notes for Seiko G1216B1N000.
;   Here is how the AT90S2313 is connected to the display:

;Port B is connected to the 8 bit data input/output on the display.
;Port D, bits 0 and 1 are not available as the corresponding pins are used for the UART.
;   these bits should be written as zero.
;Port D, bit 2 is connected to the R/W input on the display. This is
;   more of a data direction and mode pin.
;   Actual writes are clocked by the Enable pin.
;Port D, bit 3 is connected to the D/I (Data/Instruction) pin on the display.
;   Low signifies instruction on the data port.
;Port D, bit 4 is connected to the enable input on the display. Instructions and
;   data are clocked into the display on the negative edge.
;Port D, bit 5 is to the RESET input on the display. Low resets the display.
;Port D, bit 6 is connected to CS1 on the display and to the input of an inverter,
;   the output of which, drives CS2. It is therefore a right side/left side
;   select signal. Low for left side, high for right side.

;I/O Pin assignment
.equ   Enable           = $04

;Display control codes
.equ   instructionleft   = $30
.equ   instructionright  = $70
.equ   writedataleft     = $38
.equ   writedataright    = $78
.equ   readstatusleft    = $34
.equ   readstatusright   = $74

;Displays Instructions
.equ   displayon         = $3F
.equ   displayoff        = $3E
.equ   XtoZero           = $B8
.equ   YtoZero           = $40
.equ   Slinetozero       = $C0

;Flag bit assignments for register YH
;YH bit 0 if = 1 then in binary mode for serial data, if 0, ASCII-HEX mode
;YH bit 1 if = 1 then adds dot without erasing 8 bits in column (read-modify-write; slow)

;UART clock parameters
.equ   clock = 4000000      ;clock frequency
.equ   baudrate = 19200    ;choose a baudrate
.equ   baudconstant = (clock/(16*baudrate))-1

;Register Assignments

.def   temp              = r16
.def   xtarget           = r17
.def   ytarget           = r18
.def   dispdata          = r19
.def   temp2             = r20

.def   delayreg          = r23
.def   delay2            = r24
.def   inchar            = r25
.def   outchar           = r26

.org   $00
```

```

rjmp    reset                ;Reset handle

reset:  rjmp    init          ;Start init.

init:

    ldi    temp,ramend ;low(ramend)
    out    spl,temp        ;Set the stack pointer
    ldi    temp,baudconstant
    out    ubrr,temp        ;Load UART bard rate
    sbi    ucr,txen         ;enable UART xmit
    sbi    ucr,rxen         ;Enable UART receiver

    ldi    YH,$00          ;Clear all flags

    rcall   waitlongtime    ;Allow dispay to come on.

    ldi    temp,$FF        ;Issue reset.
    rcall   odd
    ldi    temp,$00
    rcall   opd

    ldi    temp,instructionleft ;Address left half of display
    rcall   opd

    ldi    temp,$FF
    rcall   odb
    ldi    temp,displayon    ;Turn left display half on.
    rcall   opb
    rcall   strobeE

    ldi    temp,SlinetoZero   ;Set left start line to zero.
    rcall   opb
    rcall   shortdelay
    rcall   strobeE

    ldi    temp,instructionright ;Address right half of display.
    rcall   opd

    ldi    temp,$FF
    rcall   odb
    ldi    temp,displayon    ;Turn right half on.
    rcall   opb
    rcall   strobeE

    ldi    temp,SlinetoZero   ;Set rightstart lien to zero.
    rcall   opb
    rcall   shortdelay
    rcall   strobeE

    rcall   erasedisplay      ;Fill display with zeros (clear screen).

    ldi    xtarget,$00        ;Draw A "V" on the screen. uses each row and each column.
    ldi    ytarget,$00

moredots:
    rcall   WriteDotAtXY
    inc     xtarget
    inc     ytarget
    cpi     xtarget,63
    brne    moredots

moredotsT:
    rcall   WriteDotAtXY
    dec     xtarget
    inc     ytarget
    cpi     xtarget,$FF
    brne    moredotsT

defaultloop:                ;*****Default loop****
                                ;Until a command is received, which is distinguished by the MSb (bit 7)
                                ;being set hight, receive x value, then y value, then write a dot there,
                                ;notify when the dot write is complete, and get another. When a command
                                ;is received, to interpret it.

                                ;Message format:
                                ;bit 7: 1 if command.
                                ;bit 6: msb of y address, or zero if x address

```

```

;bits 5 through 0 remaining bits of address

;Host should send x, then y, then wait for completion notification.
;IN THIS ROUTINE, THE AXES ARE CORRECTED. X IS THE HORIZONTAL AXIS (VALUE 0..127)
;AND Y IS THE VERTICAL AXIS (VALUE 0..63).
;SENSE IS CORRECTED ALSO. 0,0 IS LEFT,BOTTOM
;Get char and put into xtarget.

rcall GetASCIIOrBinary
mov temp2,temp
andi temp2,$80
brne ItsaCommand
andi temp,$7F
mov ytarget,temp

rcall GetASCIIOrBinary ;Get char and put into ytarget.
mov temp2,temp
andi temp2,$80
brne ItsaCommand
andi temp,$3F
ldi temp2,$3F
sub temp2,temp
mov xtarget,temp2
;

rcall WriteDotAtXY ;Write dot.
ldi temp,$2A ;Notify write is complete.
rcall emitchar

rjmp defaultloop

```

```

GetASCIIOrBinary: ;Get byte value from host to temp. If YH bit 0 is set, gets binary
;from host. If YH bit 0 is set, then gets ASCII-HEX from host.

rcall recvchar
sbrc YH,0
ret ;Binary mode - return after first byte

cpi temp,$30
brmi GetASCIIOrBinary;If comma, ignore and use next char instead
mov temp2,temp ;Get second char and make binary. No error checking.
rcall recvchar

;If comma, out of sync - start receiving byte over
cpi temp,$30
brmi GetASCIIOrBinary

sbrc temp2,6 ;convert high byte
subi temp2,$f7 ;if bit 6 is set, add $09
andi temp2,$0F

sbrc temp,6 ;convert low byte
subi temp,$f7 ;if bit 6 is set, add $09
andi temp,$0F

lsl temp2 ;combine them
lsl temp2
lsl temp2
lsl temp2
or temp,temp2
ret

```

;convert ascii in inbyteh,inbytel to byte in inbytel

```

ItsaCommand:
cpi temp,$80 ;Command $80 erase display and reset
breq ClearAndReset
cpi temp,$81 ;Command $81 switch to ASCII mode
breq SwitchToAscii
cpi temp,$82 ;Command $82 switch to binary mode
breq SwitchToBinary
cpi temp,$83 ;Command $83 restart controller
brne drs1
rjmp init

drs1:
; cpi $84
; breq ?????

ldi temp,$21 ;Else, Nack command and reset
rcall emitchar ;Exclamation mark is printable Nack
rjmp defaultloop

```

```

SwitchToAscii:
    andi    YH,$FE
    rjmp    defaultloop

SwitchtoBinary:
    ori     YH,$01
    rjmp    defaultloop

ClearAndReset:                ;Clear display and reset default interperetation loop (can be used to syncn)
    rcall   erasedisplay
    rjmp    AckAndReset

AckAndReset:
    ldi     temp,$2A            ;Send asterisk, which is printable ack
    rcall   emitchar           ;and go back to start of default loop.
    rjmp    defaultloop

erasedisplay:
    ldi     dispdata,$00
    ldi     xtarget,$00
anotherx:
    ldi     ytarget,$00
anothery:
    rcall   WriteByteAtXY
    inc     ytarget
    cpi     ytarget,128
    brne    anothery
    inc     xtarget
    cpi     xtarget,8
    brne    anotherx
    ret

WriteDotAtXY:                  ;Write a dot at X Y location
                                ;(0<=X<=63, 0<=Y<=127.
                                ;Enter with X in xtarget and
                                ;Y in ytarget (nothing to do with Y but pass it along.
    push    xtarget            ;Save xtarget on stack for later
    push    ytarget
    andi    xtarget,$3F
    andi    ytarget,$7F
    mov     temp,xtarget       ;Make display data from 3 lsb of xtarget
    andi    temp,$07
    rcall   bitpositiontoormask
    mov     dispdata,temp
                                ;Make X page address from xtarget. Put in xtarget.
    lsr     xtarget
    lsr     xtarget
    lsr     xtarget
    andi    xtarget,$07
    rcall   WriteByteAtXY
    pop     ytarget
    pop     xtarget
    ret

WriteByteAtXY:                 ;Write 8 bits at X and Y locaton per data sheet
    sbi     portd,2            ;Put R/W on display in Read to
    ldi     temp,instructionleft ;prevent ghost right write in column 1.
    mov     temp2,ytarget
    andi    temp2,$40
    breq    notrights
    ldi     temp,instructionright
notrights:
    rcall   opd
    mov     temp,xtarget
    ldi     temp2,XtoZero
    add     temp,temp2
    rcall   opb
    rcall   strobeE
    mov     temp,ytarget
    andi    temp,$3F
    ldi     temp2,YtoZero
    add     temp,temp2
    rcall   opb
    rcall   strobeE
notright:
    sbi     portd,3            ;Make this a data write.
    rcall   shortdelay
    mov     temp,dispdata
    rcall   opb

```

```

        rcall  strobeE
        ldi    temp,$00                ;Prevent glitching starting line register
        rcall  opb
        ret

odd:
        out    ddrd,temp
        ret

opd:
        out    portd,temp
        rcall  shortdelay
        ret

odb:
        out    ddrb,temp
        ret

opb:
        out    portb,temp
        rcall  shortdelay
        ret

;

strobeE:
;    rcall  shortdelay
;    cbi    portd,4
;    rcall  shortdelay
;    sbi    portd,4
;    ret

bitpositiontoormask:    ;Convert bit position (0..7) to OR mask value
                        ;Enter with bit position in temp, Exit with
                        ;mask in temp
        ldi    temp2,$07
        cp     temp2,temp
        brmi   error
        ldi    temp2,$01        ;Put 0000 0001 pattern into temp for shifting
rotatesomemore:
        cpi    temp,$00
        breq   finishedshifting
        lsl    temp2
        dec    temp
        rjmp   rotatesomemore
finishedshifting:
        mov    temp,temp2
        ret

error:
        ;Report error - for now, just return.
        ldi    temp,$21          ;Else, Nack command and reset
        rcall  emitchar          ;Exclamation mark is printable Nack
        ret

emitchar:
        ;Send byte in temp to terminal
        sbis   usr,udre          ;wait until the register is cleared
        rjmp   emitchar
        out    udr,temp          ;send the byte
        ret                    ;go back

recvchar:
        ;Receive a byte from the terminal and put into temp
        sbis   usr,rxrc          ;Wait for byte to be received
        rjmp   recvchar
        in     temp,udr          ;Read byte
        ret

shortdelay:
        ldi    delayreg,$04
delaymore:
        dec    delayreg
        brne   delaymore
        ret

longdelay:
        ldi    delayreg,$00
        ldi    delay2,$00
delaymore2:
        dec    delayreg

```

```
brne    delaymore2
dec     delay2
brne    delaymore2
ret
```

```
waitlongtime:
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
rcall   longdelay
ret
```

;http://projects.cappels.org/

;[HOME](#)